National College of Ireland

Higher Diploma in Science in Computing

HDCSDEV_INT

TABA Assignment – May 2025

Algorithms and Advanced Programming

Submission Date: May 19th, 2025

Release Date: May 16th, 2025

Student Name: Guilherme Silveira

Student ID: 23413271

Lecturer: Dr. William Clifford

Assignment Contents:

- **Question 1: Staff Data Application**
- **Question 2: Data Structure Runtime Analysis**
- **Question 3: Multithreading and Exception Handling**
- **Question 4: Merge Sort and Anomaly Detection**

Question 1 – Staff Data Application

This application processes staff information (staff.csv) using object serialization, recursive file reading, sorting (using Merge Sort), and multithreading.

a) Read CSV with BufferedReader and Serialize Data

This part of the program reads data from `Staff.csv` using `BufferedReader`, creates `Staff` objects from each row, and serializes the resulting array to a file called `Staff.dat` using `ObjectOutputStream`.

The class `StaffReader.java` opens the CSV file with `BufferedReader` and skips the header line.

```
19          // Open CSV file using BufferedReader
20          br = new BufferedReader(new FileReader(csvFile));
21          br.readLine(); // Skip the header line
```

Each line is split using `split(",")`, and values are trimmed and parsed into appropriate types. A `Staff` object is created using these values and added to a fixed-size `Staff[]` array.

```
26      // Read each line, convert to Staff object and add to list
27      while ((line = br.readLine()) != null) {
28          String[] values = line.split(regex:","); // Split the values within a line
29
30          // Assign values to variables
31          int empNo = Integer.parseInt(values[0].trim());
32          String firstName = values[1].trim();
33          String lastName = values[2].trim();
34          String department = values[3].trim();
35          double wage = Double.parseDouble(values[4].trim());
36          double pCompRate = Double.parseDouble(values[5].trim());
37
38          // Create Staff object and add to list
39          Staff staff = new Staff(empNo, firstName, lastName, department, wage, pCompRate);
40          staffArray[index++] = staff; // Add Staff object to the list
41      }
```

After reading all lines, the array is serialized using `ObjectOutputStream` to a file named `Staff.dat`.

```
43          // Serialize the list of Staff objects to a file
44          ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(serializedFile));
45          oos.writeObject(staffArray);
46          oos.close();
```

b) Recursive Deserialization

This part of the program reads the serialized `Staff.dat` file, reconstructs the original `Staff[]` array, and recursively prints object to the console.

The class `StaffDeserializer.java` uses `ObjectInputStream` to read the contents of the `Staff.dat` file. The deserialized object is cast to a `Staff[]` array.

```
10          // Read serialized file
11          // Open file and create an object input stream to read objects
12          ObjectInputStream ois = new ObjectInputStream(new FileInputStream(serializedFile));
13
14          // Read the array of staff objects from the file and cast it
15          Staff[] staffArray = (Staff[]) ois.readObject();
16          ois.close();
```

A recursive method, `printStaff(…)`, is used to print each object one by one.

```
28      // Recursive method to print the array
29      public static void printStaff(Staff[] array, int index) {
30          if (index >= array.length) {
31              return; // Base Case: Stop when it reaches the end
32          }
33          System.out.println(array[index]); // Print current staff
34          printStaff(array, index + 1); // Recursive call with next index to print the next Staff object
35      }
```

c) Sorting by Column Using Merge Sort

This part of the program allows the user to choose a column from the staff data to sort by. The selected field is then used to sort the deserialized array using a manual implementation of the Merge Sort algorithm.

The class `StaffSorter.java` begins by deserializing the array of `Staff` objects from `Staff.dat` using `ObjectInputStream`. A GUI prompt (`JOptionPane`) lets the user select the column to sort by.

```
11          // Deserialize the list
12          ObjectInputStream ois = new ObjectInputStream(new FileInputStream(serializedFile));
13          Staff[] staffArray = (Staff[]) ois.readObject();
14          ois.close();
15
16          // Ask user for sorting preference
17          String input = JOptionPane.showInputDialog(
18                  message:"Sort by:\n\n1 - Employee Number\n2 - First Name\n3 - Last Name\n4 - Department\n5 - Wage\n6 - Project Completion Rate");
19          int choice = Integer.parseInt(input);
```

The array is sorted using a custom `mergeSort(…)` method, which calls `merge(…)` and, finally, `compareStaff(…)` to compare objects based on the selected field.

```
34      public static void mergeSort(Staff[] array, int lowerB, int upperB, int column) {
35          // Making sure that there is at least two elements in the list
36          if (lowerB + 1 < upperB) {
37              // Spliting the list by two and finding the middle element
38              int mid = (lowerB + upperB) / 2;
39              // Sorting the left side of the list recursively
40              mergeSort(array, lowerB, mid, column);
41              // Sorting the right side of the list recursively
42              mergeSort(array, mid, upperB, column);
43              // Merging both sides
44              merge(array, lowerB, mid, upperB, column);
45          }
46      }
```

```java
public static void merge(Staff[] array, int lowerB, int mid, int upperB, int column) {
    // Initializing two pointers to the lowest index of each side (left and right)
    int i = lowerB, j = mid;
    // Creating a temporary list that will store the sorted values
    Staff[] temp = new Staff[upperB - lowerB];
    int k = 0;

    // Merging the two halves while comparing using compareStaff()
    // This loop will stop when one of the sides is fully merged
    while (i < mid && j < upperB) {
        if (compareStaff(array[i], array[j], column) <= 0) {
            temp[k] = array[i];
            i++;
        } else {
            temp[k] = array[j];
            j++;
        }
        k++;
    }

    // If there are leftover elements, which are already sorted, in one of the
    // halves, we need to copy them into the temp ArrayList
    // Checking leftover elements in the left side and adding them to temp
    while (i < mid) {
        temp[k] = array[i];
        i++;
        k++;
    }
    // Checking leftover elements in the right side and adding them to temp
    while (j < upperB) {
        temp[k] = array[j];
        j++;
        k++;
    }

    // We need to copy the elements from temp[] back to the original array in the
    // correct range
    // This will put the elements in the right order into the original list
    for (int t = 0; t < temp.length; t++) {
        array[lowerB + t] = temp[t];
    }
}
```

```java
private static int compareStaff(Staff a, Staff b, int column) {
    switch (column) {
        case 1:
            return Integer.compare(a.getEmpNo(), b.getEmpNo());
        case 2:
            return a.getFirstName().compareToIgnoreCase(b.getFirstName());
        case 3:
            return a.getLastName().compareToIgnoreCase(b.getLastName());
        case 4:
            return a.getDepartment().compareToIgnoreCase(b.getDepartment());
        case 5:
            return Double.compare(a.getWage(), b.getWage());
        case 6:
            return Double.compare(a.getPCompRate(), b.getPCompRate());
        default:
            return 0;
    }
}
```

The sorted list is printed to the console.

```java
// Print sorted list
for (Staff staff : staffArray) {
    System.out.println(staff);
}
```

d) Multithreaded Sorting and File Output

This section of the program creates six threads, each responsible for sorting a copy of the staff array using a different column. After sorting, each thread writes the result to a separate output file.

`StaffThreadSorter.java` deserializes the staff data and starts 6 `StaffSorterThread` threads – one for each of the six columns.

```
10            ObjectInputStream ois = new ObjectInputStream(new FileInputStream(serializedFile));
11            Staff[] staffArray = (Staff[]) ois.readObject();
12            ois.close();
13
14            // Create and start threads
15            StaffSorterThread[] threads = new StaffSorterThread[6];
16
17            for (int col = 1; col <= 6; col++) {
18                threads[col - 1] = new StaffSorterThread(staffArray, col);
19                threads[col - 1].start();
20            }
```

Each thread:

- Sorts the array using the existing `mergeSort()` method
- Writes the result to a file named `sortedStaff_C{column}.csv`

```
10    // Constructor
11    public StaffSorterThread(Staff[] original, int column) {
12        // Copy the original array into data
13        this.data = new Staff[original.length];
14        for (int i = 0; i < original.length; i++) {
15            this.data[i] = original[i];
16        }
17        this.column = column;
18    }
19
20    @Override
21    public void run() {
22        // Sort using mergeSort
23        StaffSorter.mergeSort(data, lowerB:0, data.length, column);
24
25        // Save the sorted array to sortedStaff_C{column}.csv
26        String filename = "sortedStaff_C" + column + ".csv";
27
28        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
29            // Write header
30            writer.write(str:"empNo,firstName,lastName,department,wage,pCompRate");
31            writer.newLine();
32
33            // Write each staff row
34            for (Staff staff : data) {
35                String line = staff.toString();
36                writer.write(line);
37                writer.newLine();
38            }
```

The main thread waits for all sorting threads to complete using `join()`.

```
22            // Wait for all threads to finish
23            for (StaffSorterThread thread : threads) {
24                try {
25                    thread.join();
26                } catch (InterruptedException e) {
27                    System.out.println("Thread interrupted: " + e.getMessage());
28                }
29            }
```

Question 2

a) Find the largest value

Unsorted Array: One must look at every single element to find the largest and there is no guarantee that it is at the end. **Time Complexity: O(n)**
Sorted Array: Since it is sorted (assuming ascending), the largest element is at the last index. So, it can be accessed directly by array[n – 1]. **Time Complexity: O(1)**
Linked List: Even if it is sorted, it is not possible to jump to the last node – one must traverse it. So, other elements must be checked. **Time Complexity: O(n)**

b) Retrieving a value from a specific index

Unsorted and Sorted Arrays: Since they have direct indexes, any index can be accessed instantly, which leads to **Time Complexity: O(1)**.
Linked List: Random access is not supported. Therefore, to get the value of a specific element, one must traverse them node by node. **Time Complexity: O(n)**.

c) Removing an item from the list

Unsorted Array: To remove an element, it is necessary to find it (O(n)) and then shift all elements after it one position to the left (O(n)). **Worst-case time complexity: O(n)**
Sorted Array: If the index is known, removing still requires shifting, which is O(n). If the value is known, binary search can be used to find the index in O(log n), but the overall operations is still O(n) due to shifting. **Overall: O(n).**
Linked List: One must traverse the list to the node just before the one to be removed (O(n)) and then change the pointer O(1). **Time Complexity: O(n)**.

d) Adding an item (position unspecified but there will be one more bit of data than there was before).

Unsorted Array: Easiest case – add the new item to the end of the array. If the array has space, this is O(1). If it is full, it is necessary to allocate a bigger array, copying all elements to it (O(n)). So, in general: **Time Complexity: Best case – O(1); Worst case – O(n)**.

Sorted Array: Find the correct insertion point (binary search if needed), O(log n). Then, shift the elements to make space (**O(n)**). **Time Complexity: O(n)**.

Linked List: Since the position is unspecified, it can be just inserted at the head or tail (O(1)). If inserting at a specific index, it is necessary to traverse it (O(n**)**). **Time Complexity: O(1)**.

e) Checking if the data structure contains duplicates

Unsorted Array: Each item must be compared with all others which leads to a nested loop. **Time Complexity: O(n^2)**.

Sorted Array: Duplicates can be checked in one pass by comparing each element with the next one. **Time Complexity: O(n)**.
Linked List: No random access or sorted order, unless it is a sorted list. It requires a nested traversal. **Time Complexity: O(n^2)**.

f) How could you test the run-time of these algorithms and their respective data structures in a real system on your own personal machine? You can provide examples as well as a description if this helps.

To test the runtime of algorithms and data structures, Java's built-in timing functions can be used to measure how long a specific operation takes. The most common method is using System.nanoTime() or System.currentTimeMillis() to capture the start and end times of the code being tested.

Using System.nanoTime(): This method gives high resolution timing and is ideal for measuring algorithm performance:

```
long startTime = System.nanoTime();
// algorithm to test
long endTime = System.nanoTime();
long duration = endTime - startTime;

System.out.println("Time taken: " + duration + " nanoseconds");
```

This approach can be used to test operations such as searching, sorting, or inserting elements in different data structures.

Comparative Testing: To compare different structures (e.g., arrays vs linked lists), one could run the same operation on each with identical data sizes and measure the difference in execution time. It is important to repeat the test multiple times and average the results to get reliable measurements, since execution time can vary depending on system load.

Testing with larger inputs: To clearly observe differences in time complexity (such as O(n) vs O(n^2), it is a good idea to run the tests using increasingly larger inputs, such as 1,000, 10,000, and 100,000 elements. As the input size grows, inefficiencies in algorithms with higher time complexity become more visible.

In summary, the runtime of data structure operations can be tested using Java's timing functions, structured test cases, and repeated measurements. It allows us to evaluate the practical performance of algorithms in different scenarios.

Question 3

a) What is the concept of starvation with respect to multi-threading? Provide an example of where starvation might occur while using multi-threaded programs.

Starvation is a condition in multi-threading where a thread is prevented from accessing resources because other threads are continuously given priority. As a result, the starved thread may never get a chance to execute, even though it is ready. (GeeksForGeeks, 2024)

For example, in a system where one thread holds a lock repeatedly and others are trying to acquire it, the waiting threads might be starved if the scheduler continuously favours the same thread. Starvation can also occur when using thread priorities. Lower priority threads may never be scheduled if higher-priority threads keep the CPU busy.

b) What is the concept of deadlock with respect to multi-threading? Provide an example of a deadlock involving at least three threads.

Deadlock is a situation in multi-threading where two or more threads are blocked forever, each waiting for a resource that is held by another thread in the cycle. This usually happens when multiple threads acquire locks in different orders and form a circular dependency, where none of them can continue execution. (Tutorials Point, 2025)

For example, consider a scenario with three threads:

- Thread 1 locks resource A and waits for resource B
- Thread 2 locks resource B and waits for resource C
- Thread 3 locks resource C and waits for resource A

In this case, all threads are waiting on each other in a cycle, and none of them can proceed. This results in a deadlock.

c) Can you explain the exception handling chain? In this answer consider where exceptions are thrown and the various levels they can be caught.

The exception handling chain describes how exceptions are passed through the call stack when they are thrown. If an exception occurs in a method and there is no matching catch block in that method, the exception is passed to the calling method. This process continues until the exception is caught or it reaches the top level of the program. (GeeksForGeeks, 2025)

If no method in the chain catches the exception, the program terminates, and the JVM prints an error message with a stack trace. This trace shows the order in which the methods were called and helps identify where the exception happened.

This chain allows developers to choose where in the program it is most appropriate to handle certain types of errors.

d) What is a custom exception? Can you briefly describe how you would create your own custom exception (you may use java or pseudocode to aid with this description).

A custom exception is a user-defined exception created to represent specific error conditions that are not covered by Java's built-in exceptions. It allows the developer to make error handling more meaningful and easier to understand.

To create a custom exception in Java, we define a class that extends the Exception class (or RuntimeException for unchecked exceptions). We can then throw it using the throw keyword when a specific error occurs.

Below is an example from a code I wrote for Data Structures, when implementing a method called findMax() to find the maximum value in a Binary Tree.

1 – The method throws an exception called TreeEmptyException() when the tree is empty.

```
61      public T findMax() {
62          if (isEmpty()) {
63              throw new TreeEmptyException();
64          } else {
65              return findMax(root);
66          }
67      }
```

2 – This exception is implemented in a separate class and returns a message indicating that the tree has no nodes.

TreeEmptyException.java > ...

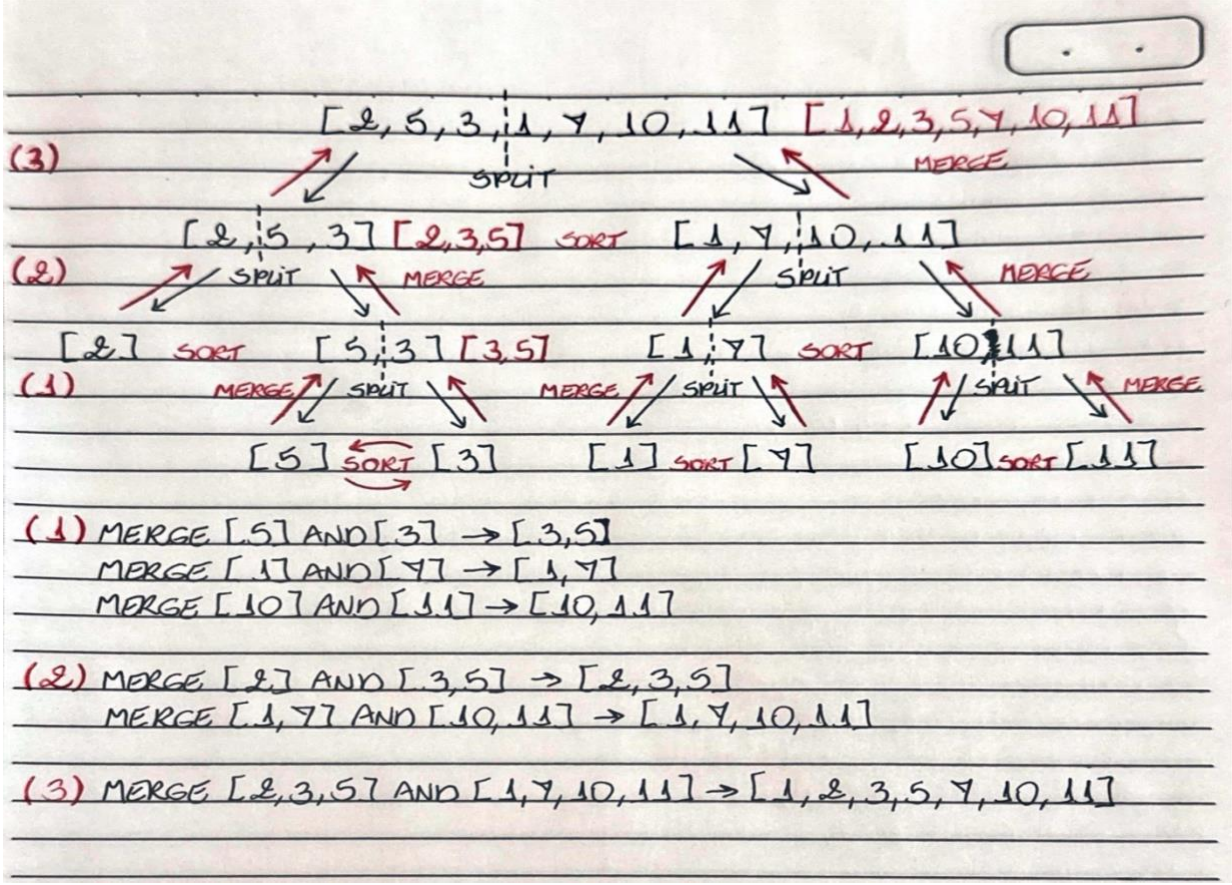
```
1   public class TreeEmptyException extends RuntimeException {
2       public TreeEmptyException() {
3           super(message:"There are no elements in the tree");
4       }
5   }
6
```

Question 4

a) Describe the merge sort sorting algorithms and show step by step how they would sort the numbers below. Do this by drawing a tree structure to represent the recursive call stack:

Merge Sort is a divide and conquer algorithm that splits the array recursively until each element is in its own sub-array, then merges the sub-arrays in sorted order. It is efficient and stable, with a time complexity of O(n log n).

The image below represents a Merge Sort tree for the given example.



Merge Sort Steps:

Initial Input
The algorithm begins with the array: [2, 5, 3, 1, 7, 10, 11]

Step 1 – Divide
The array is split into two halves:

- Left: [2, 5, 3]
- Right: [1, 7, 10, 11]

Step 2 – Recursively divide left half

- [2, 5, 3] is split into [2] and [5, 3]
- [5, 3] is split into [5] and [3]

Step 3 – Merge left half

- [5] and [3] are merged into [3, 5]
- [2] and [3, 5] are merged into [2, 3, 5]

Step 4 – Recursively divide right half

- [1, 7, 10, 11] is split into [1, 7] and [10, 11]
- [1, 7] is split into [1] and [7]
- [10, 11] is split into [10] and [11]

Step 5 – Merge right half

- [1] and [7] are merged into [1, 7]
- [10] and [11] are merged into [10,11]
- [1, 7] and [10,11] are merged into [1, 7, 10, 11]

Step 6 – Final merge

The two sorted halves [2, 3, 5] and [1, 7, 10, 11] are merged.

The final sorted array is: [1, 2, 3, 5, 7, 10, 11]

b) Anomaly Detection Code

The Java application reads a log.txt file, validates each row according to predefined anomaly scenarios, and prints or stores relevant information as required. If any invalid data is found, it is written to a separate error.txt file using custom exception handling.

Part (i) – Reading and Printing log.txt

The method `printLogFile(String fileName)` reads the file line by line by calling the helper method `readLines(String fileName)`, which uses `BufferedReader`. Then, it prints each line to standard output. This satisfies the requirement of displaying the file content.

```
22      // Print each line from the file
23      public static void printLogFile(String fileName) {
24          String[] lines = readLines(fileName);
25
26          for (String line : lines) {
27              if (line != null) {
28                  System.out.println(line);
29              }
30          }
31      }
32
33      // Helper method to read lines
34      private static String[] readLines(String fileName) {
35          String[] lines = new String[100];
36          int count = 0;
37
38          try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {
39              String line;
40              while ((line = reader.readLine()) != null && count < lines.length) {
41                  lines[count++] = line;
42              }
43          } catch (IOException e) {
44              System.out.println("Error reading file" + e.getMessage());
45          }
46          return lines;
47      }
```

Part (ii) – Detecting Anomalies with Custom Exceptions

The method `processLogFile(String fileName)` validates each line (excluding the header) by calling the helper method `validateLines(String line)`. It checks for the following anomalies:

1. **Invalid year format** (year must start with 4 digits)
2. **Invalid country code** (phone must start with +353)
3. **Missing or invalid ID** (ID must be a positive integer)
4. **Missing currency symbol** (fee must start with €, £ or $)

Each of these checks throws a specific custom exception.

```java
    // Validate each line (skipping the header) and store invalid ones
    public static void processLogFile(String fileName) {
        String[] lines = readLines(fileName);
        boolean isFirstLine = true; // to skip header

        for (String line : lines) {
            // Prevent crashing on empty String[] elements
            if (line == null) {
                continue;
            }
            // Skips the header line from being validated
            if (isFirstLine) {
                isFirstLine = false;
                continue;
            }

            try {
                // Validate each line for anomalies
                validateLine(line);
            } catch (InvalidYearException | InvalidPhoneException | MissingIdException
                    | InvalidCurrencyException e) {
                // Print the error to the console
                System.out.println("Anomaly detected: " + e.getMessage());

                // Save the invalid line to the array (up to 100)
                if (invalidCount < invalidLines.length) {
                    invalidLines[invalidCount] = line;
                    invalidCount++;
                }
            }
        }
    }
```

```java
    // Split the line into columns and check 4 types of anomaly
    private static void validateLine(String line)
            throws InvalidYearException, InvalidPhoneException, MissingIdException, InvalidCurrencyException {
        String[] columns = line.split(regex:",");

        // Skip empty or malformed lines
        if (columns.length < 6) {
            return;
        }

        String id = columns[0].trim();
        String phone = columns[3].trim();
        String fee = columns[4].trim();
        String date = columns[5].trim();

        // Scenario 1: Date must start with 4 digits followed by '/'
        if (!date.matches(regex:"^\\d{4}/.*")) {
            throw new InvalidYearException(message:"Date year must start with 4 digits");
        }

        // Scenario 2: Phone must start with +353
        if (!phone.startsWith(prefix:"+353")) {
            throw new InvalidPhoneException(message:"Invalid country code in phone number");
        }

        // Scenario 3: ID must exist and be an integer > 0
        try {
            if (id.isEmpty() || Integer.parseInt(id) <= 0) {
                throw new MissingIdException(message:"Invalid or missing ID");
            }
        } catch (NumberFormatException e) {
            throw new MissingIdException(message:"ID is not a valid integer");
        }

        // Scenario 4: Fee must start with €, £ or $
        if (!(fee.startsWith(prefix:"€") || fee.startsWith(prefix:"£") || fee.startsWith(prefix:"$"))) {
            throw new InvalidCurrencyException(message:"Fee does not start with a valid currency symbol.");
        }

    }
```

Below are the custom exception classes:

```java
// CUSTOM EXCEPTION CLASSES
// Custom exception for invalid year in date
public static class InvalidYearException extends Exception {
    public InvalidYearException(String message) {
        super(message);
    }
}

// Custom exception for invalid phone number (wrong country code)
public static class InvalidPhoneException extends Exception {
    public InvalidPhoneException(String message) {
        super(message);
    }
}

// Custom exception for missing or invalid ID
public static class MissingIdException extends Exception {
    public MissingIdException(String message) {
        super(message);
    }
}

// Custom exception for invalid currency format in the fee
public static class InvalidCurrencyException extends Exception {
    public InvalidCurrencyException(String message) {
        super(message);
    }
}
```

Part (iii) – Writing invalid Entries to error.txt

The method `writeErrorsToFile(String filename)` iterates through all invalid entries collected during validation and writes them to the error.txt file using `BufferedWriter`.

```java
// Write all collected invalid lines to a file (error.txt)
public static void writeErrorsToFile(String fileName) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
        // Write each invalid line
        for (int i = 0; i < invalidCount; i++) {
            writer.write(invalidLines[i]); // Write the line
            writer.newLine(); // Add a new line after each entry
        }
        // Print confirmation
        System.out.println("Invalid lines written to " + fileName);
    } catch (IOException e) {
        // Handle file writing errors
        System.out.println("Error writing to file: " + e.getMessage());
    }
}
```

Bibliography

GeeksForGeeks, 2024. *Starvation and Aging in Operating Systems*. [Online]
Available at: https://www.geeksforgeeks.org/starvation-and-aging-in-operating-systems/
[Accessed 19 May 2025].

GeeksForGeeks, 2025. *Exception Propagation in Java*. [Online]
Available at: https://www.geeksforgeeks.org/exception-propagation-java/
[Accessed 19 May 2025].

Tutorials Point, 2025. *Introduction of Deadlock in Operating System*. [Online]
Available at:
https://www.tutorialspoint.com/operating_system/introduction_to_deadlock_in_operating_system.htm
[Accessed 19 May 2025].